Aventure
partie 7

Remarque

— pour chaque classe référez vous à la javadoc
— limitez vous à ce que vous avez vu en cours
— méthode toString() à faire systématiquement

1 La classe `JoueurHumain`

1. Donner un ordre d'action à un joueur humain passe par la méthode `setOrdre` qui prend une simple chaîne de caractères (par exemple : ouvrir porte1, prendre couteau, ...). Cet ordre doit être stocké dans le joueur
2. Définissez des méthodes privées pour chaque ordre que le joueur humain pourra exécuter

```
void commandePrendre(String nomObjet)
     throws ObjetAbsentDeLaPieceException, ObjetNonDeplacableException
void commandePoser(String nomObjet)
     throws ObjetNonPossedeParLeVivantException
void commandeFranchir(String nomPorte)
     throws PorteFermeException, PorteInexistanteDansLaPieceException
void commandeOuvrirPorte(String nomPorte)
     throws ActivationException, PorteInexistanteDansLaPieceException
void commandeOuvrirPorte(String nomPorte, String nomObjet)
     throws ActivationException, PorteInexistanteDansLaPieceException,
            ObjetNonPossedeParLeVivantException
```

3. La méthode `executer` **est pour l'instant vide**, elle devra dans le prochain TP analyser l'ordre (la chaîne de caractère stockée) et appeler la méthode privée correspondante.

2 Les interfaces `Lecteur` & `Enregistreur`

Créez les 2 interfaces.

3 La classe `LecteurDescription`

L'objectif de cette classe est de lire un flux contenant une description du jeu d'aventure. On considère que le fichier ne contient pas d'erreur.

Exemple :

```
Reader reader = new FileReader("exemplesimulation1.aa");
Lecteur lecteur = new LecteurDescription(reader);
Monde monde = lecteur.getMonde();
// faites les affichages nécessaires pour vérifier votre travail
```

Conseils de développement

— utilisez un `Scanner`
— utilisez un `switch-case` en fonction du premier mot de chaque ligne
— créez des méthodes privées appelées dans chaque `case`
— développez dans le même ordre que le fichier

Si vous utilisez le fichier de description donné, vous pourrez utiliser les tests unitaires fournis pour vérifier votre travail. Positionnez le fichier de description dans votre projet au même niveau que `src` et `srctest`.

4 La sérialisation

`EnregistreurSerialisation`

L'objectif est de pouvoir écrire des objets sérialisés dans le flux passé au constructeur.

Exemple :

```
OutputStream fic = new FileOutputStream(nomFichier);
Enregistreur enr = new EnregistreurSerialisation(new ObjectOutputStream(fic));
enr.enregistrer(monde);
fic.close();
```

Vous pouvez « regarder » le contenu du fichier pour voir si cela ressemble à une sérialisation de vos données.

`LecteurSerialisation`

L'objectif est de pouvoir lire des objets sérialisés à partir du flux passé au constructeur.

Exemple :

```
InputStream fic = new FileInputStream(nomFichier);
Lecteur lec = new LecteurSerialisation(new ObjectInputStream(fic));
fic.close();
```

```
Monde m = lec.getMonde();
// faites les affichages nécessaires pour vérifier votre travail
```

5 La classe `Main`

Elle contient la méthode `main` qui présente un menu du type :

```
 --- Menu ---
1/ jouer
2/ charger un fichier de description
3/ sauver la partie actuelle
4/ charger une partie
5/ quitter
```

À vous de faire fonctionner les points 2, 3, 4, 5 de ce menu.